

Experiment-01

Aim:

- A. Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:
- "Low Salary": All monthly salaries strictly less than \$20,000.
 - "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
 - "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.
- B. Design an Employee Management System database architecture by implementing the following operations sequentially:
- Create and populate structural tracking tables for Departments, Employees, and Salaries.
 - Query the system using multi-table Joins to pull clear department breakdowns.
 - List all remaining active employees whose personal salary exceeds the overall company-wide average salary.
- C. A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`. Every customer must choose exactly 3 different toppings to prepare a pizza.
- Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.
- Requirements:
- Every pizza must contain exactly 3 different toppings.
 - The same topping cannot appear more than once in a pizza.
 - Different arrangements of the same toppings should be considered the same pizza.
- Example:
- Chicken, Pepperoni, Sausage
Pepperoni, Chicken, Sausage
Sausage, Chicken, Pepperoni
- These are all the same pizza, so only one should appear.
- Display the topping names separated by commas.
 - Display the total ingredient cost of each pizza.
 - Sort the output:
 - First by highest total cost
 - If two pizzas have the same cost, sort them alphabetically.

D. Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements:

- Compare purchases made by the same user only.
- Ignore comparisons where the same transaction is matched with itself.
- The second purchase must occur after the first purchase.
- Same-day purchases must not be considered.
- The second purchase must occur within 7 days of the first purchase.
- Display only the unique User IDs that satisfy the above conditions.
- Sort the output in ascending order of user_id.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

A JOIN is an SQL operation used to combine rows from two or more tables based on a related column between them. Joins help retrieve meaningful information stored across multiple tables while maintaining database normalization.

The most common types of joins are:

- INNER JOIN - returns only the rows that have matching values in both tables. If a row has no corresponding match in the other table, it is not included in the result.
- LEFT OUTER JOIN - returns all rows from the left table and the matching rows from the right table. If no match exists, NULL values are returned for the right table columns.
- RIGHT OUTER JOIN - returns all rows from the right table and matching rows from the left table. If no matching record exists in the left table, NULL values are displayed.
- FULL OUTER JOIN - returns all records from both tables. Matching records are combined, while non-matching rows from either table contain NULL values.
- CROSS JOIN - returns the Cartesian Product of two tables. Every row from the first table is combined with every row from the second table.
- SELF JOIN - joins a table with itself using aliases. It is useful when rows within the same table have relationships.
- LEFT EXCLUSIVE JOIN - returns only those rows from the left table that have no matching rows in the right table.
- RIGHT EXCLUSIVE JOIN - returns only those rows from the right table that have no matching rows in the left table.

Code and Output:

A.

The image shows two screenshots of a PostgreSQL IDE interface. The top screenshot displays a SQL script for creating a table and inserting data. The bottom screenshot displays a query that categorizes accounts by salary range and shows the resulting data table.

Top Screenshot: SQL Script

```
1 CREATE TABLE Accounts (  
2     account_id INT PRIMARY KEY,  
3     income INT NOT NULL  
4 );  
5  
6 INSERT INTO Accounts (account_id, income) VALUES  
7 (3, 108939),  
8 (2, 12747),  
9 (8, 87709),  
10 (6, 91738),  
11 (7, 45169);
```

Bottom Screenshot: Query and Output

```
1 SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count  
2 FROM Accounts WHERE income < 20000  
3  
4 UNION ALL  
5  
6 SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count  
7 FROM Accounts WHERE income >= 20000 AND income <= 50000  
8  
9 UNION ALL  
10  
11 SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count  
12 FROM Accounts WHERE income > 50000;
```

Data Output Table:

	category	accounts_count
1	Low Salary	1
2	Average Sala...	1
3	High Salary	3

B.

The image shows two screenshots of a PostgreSQL query editor interface. The top screenshot displays the creation of three tables: Departments, Employees, and Salaries. The bottom screenshot displays the insertion of data into these tables.

Top Screenshot: Query Editor

Experiment_01/postgres@PostgreSQL 18

Query History

```
1 CREATE TABLE Departments (  
2     dept_id INT PRIMARY KEY,  
3     dept_name VARCHAR(50) NOT NULL  
4 );  
5  
6 CREATE TABLE Employees (  
7     emp_id INT PRIMARY KEY,  
8     name VARCHAR(50) NOT NULL,  
9     dept_id INT REFERENCES Departments(dept_id)  
10 );  
11  
12 CREATE TABLE Salaries (  
13     emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,  
14     salary INT NOT NULL  
15 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 128 msec.

Bottom Screenshot: Query Editor

Experiment_01/postgres@PostgreSQL 18

Query History

```
1 INSERT INTO Departments (dept_id, dept_name) VALUES  
2 (10, 'HR'), (20, 'IT'), (30, 'Sales');  
3  
4 INSERT INTO Employees (emp_id, name, dept_id) VALUES  
5 (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);  
6  
7 INSERT INTO Salaries (emp_id, salary) VALUES  
8 (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 58 msec.

Experiment_01/postgres@PostgreSQL 18

No limit

E

Query Query History

1 SELECT e.name, d.dept_name, s.salary
2 FROM Employees AS e
3 JOIN Departments AS d
4 ON e.dept_id = d.dept_id
5 JOIN Salaries AS s
6 ON e.emp_id = s.emp_id;

Data Output Messages Notifications

+ ↻ ⌵ 🗑️ 🔍 ⬇️ ⏪ SQL

Showing rows: 1 to 5

	name character varying (50)	dept_name character varying (50)	salary integer
1	Alice	HR	50000
2	Bob	HR	25000
3	Charlie	IT	80000
4	David	IT	40000
5	Emma	Sales	45000

Experiment_01/postgres@PostgreSQL 18

No limit

E

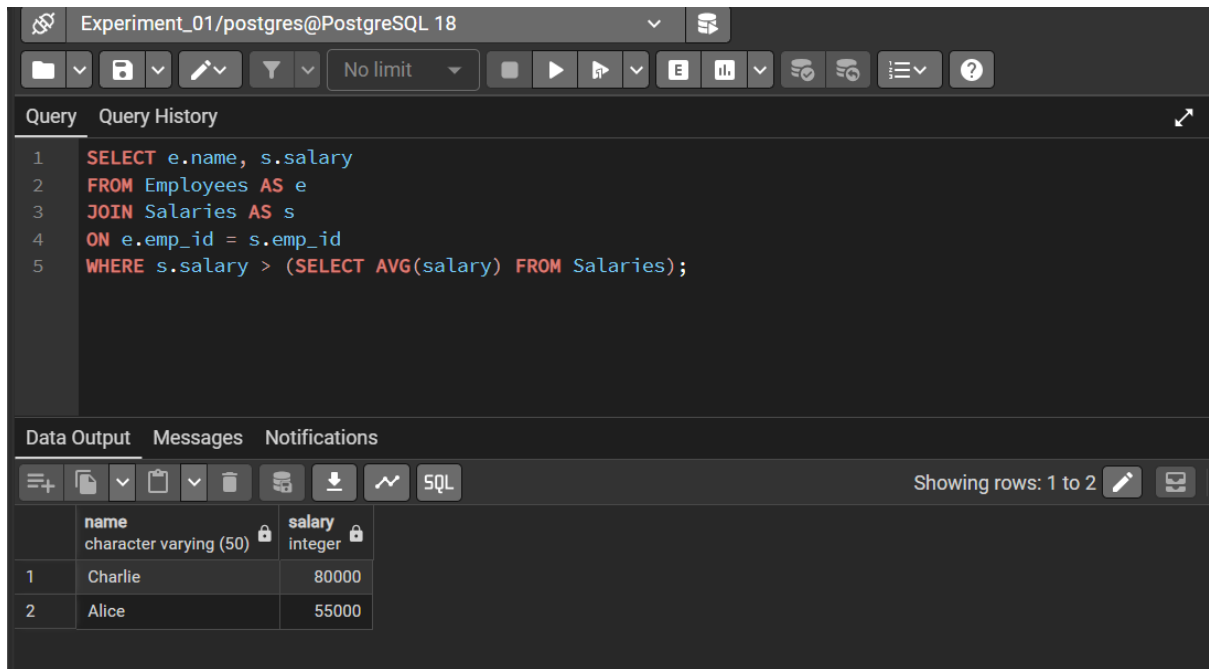
Query Query History

1 UPDATE Salaries
2 SET salary = salary * 1.10
3 WHERE emp_id IN (
4 SELECT e.emp_id
5 FROM Employees e
6 JOIN Departments d
7 ON
8 e.dept_id = d.dept_id
9 WHERE d.dept_name = 'HR'
10);

Data Output Messages Notifications

UPDATE 2

Query returned successfully in 50 msec.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_01/postgres@PostgreSQL 18'. The query editor contains the following SQL query:

```
1 SELECT e.name, s.salary
2 FROM Employees AS e
3 JOIN Salaries AS s
4 ON e.emp_id = s.emp_id
5 WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table format. The table has two columns: 'name' (character varying (50)) and 'salary' (integer). The results show two rows:

	name	salary
1	Charlie	80000
2	Alice	55000

The interface also includes a 'Messages' tab and a 'Notifications' tab. The bottom right corner shows 'Showing rows: 1 to 2'.

Note: Experiment 1 part-c and part-d were given as home assignments and are included in the home task file.

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with JOINS in PostgreSQL.
- Understood the concept of JOINS.